Application Layer Specification

Command Dispatch, Authorization, and Lock Actuation
for the NFC Smart Lock Protocol

Version 0.2 – Draft

July 24, 2026

Abstract

This document specifies the application module of the NFC smart lock protocol: the plaintext command set exchanged once a secure session is established, the current authorization policy, and the logic that actuates the physical lock mechanism. This module works as an *equal-standing peer* of the cryptographic session module (*smartlock_session_layer.pdf*), not as a layer built on top of it – the two collaborate directly (Section **??**), and neither is functional without the other. This module has no knowledge of cryptographic operations, key material, or transport/APDU framing (*smartlock_transport_layer.pdf*); it receives only plaintext byte strings that the session module has already authenticated as originating from a device holding a provisioned Ed25519 identity, and it returns plaintext byte strings for the session module to encrypt. Unlike a pure communications layer, however, this module also owns a second, independent responsibility that has nothing to do with messaging at all: driving the physical lock actuator (Section **??**). This document was split out of the previously combined *smartlock_application_layer.pdf* (v1.0), whose cryptographic content now lives in *smartlock_session_layer.pdf*; see Section **??**. It is deliberately scoped narrowly today so that upcoming work – OTA firmware update dispatch, an access control list (ACL), and an out-of-band BLE command source – can be added as clearly bounded extensions (Section **??**) rather than accreting into a single monolithic module again.

Contents

1 Scope and Layering

1.1 Purpose

This layer owns three responsibilities, and only these three:

1. **Command dispatch** – interpreting the plaintext byte string delivered by the session layer as a structured command and routing it to the correct handler (Section **??**).
2. **Authorization** – deciding whether an authenticated device is permitted to have a given command executed (Section **??**).
3. **Actuation** – driving the physical lock mechanism and reporting its resulting state (Section **??**).

It MUST NOT implement, re-implement, or depend on the internal details of Ed25519/X25519/HKDF/AES-GCM operations, nonce generation, or key storage; those are exclusively the session module's responsibility. It MUST NOT assume anything about how its plaintext arrived (NFC, or in a future revision, BLE – see Section **??**) beyond the guarantee, provided by whichever module delivered it, that the sender's long-term identity has been authenticated for this session. Note that this module is not purely a messaging consumer: Section **??** gives it a second external dependency, the physical actuator, that has no counterpart in the session module and does not route through it.

1.2 Non-Goals

This specification does not cover: cryptographic session establishment or secure messaging (*smartlock_session_layer.pdf*), APDU framing, instruction sets, or ISO-DEP timing (*smartlock_transport_layer.pdf*), NFC controller specifics (*smartlock_hardware_link_layer_pn532_spec.pdf*), provisioning workflows, or physical/mechanical design of the lock hardware itself (motor, solenoid, or deadbolt selection). Command sources other than the NFC secure session – in particular the planned BLE out-of-band channel – are explicitly deferred; see Section **??**.

2 Relationship to Other Components

This module is not a rung on a protocol ladder. It has exactly two external dependencies, of two fundamentally different kinds, and neither is "below" it:

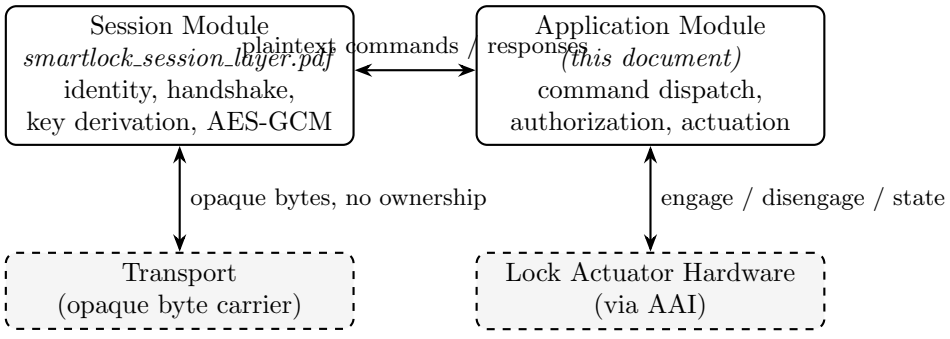

Figure 1: This module collaborates with the session module as a peer, not a downstream consumer. It also depends directly on the lock actuator hardware – a dependency the session module has no part in, and no visibility into.

- **The session module** (Ed25519 / X25519 / HKDF / AES-GCM handshake and secure messaging, *smartlock_session_layer.pdf*) hands this module authenticated plaintext and accepts plaintext back for encryption. It has no knowledge of command content, and this module has no knowledge of how that plaintext was authenticated or encrypted. Section **??** defines this collaboration in full; it is bidirectional, not a call from a higher layer into a lower one.
- **The lock actuator hardware** (Section **??**) is this module's other, unrelated dependency: physical GPIO/motor control that the session module never touches and has no reason to know exists. This is precisely why the two are peers rather than a stack – the application module's job is not simply "the layer that consumes session-module output"; roughly half of what it does (Section **??**) has nothing to do with communications at all.
- **Transport** (APDU framing, instruction set, ISO-DEP timing; *smartlock_transport_layer.pdf*) and the **hardware/link layer** (concrete NFC controller driver; *smartlock_hardware_link_layer_pn532_spec.pdf*) are real dependencies of the session module, not of this one. This module never sees an APDU, a nonce, or a ciphertext byte; those stay entirely within the session module's side of Figure **??**. The session module, in turn, does not own or control the transport's state machine or timing – it only hands the transport opaque bytes, the same way this module hands the session module opaque plaintext.

A device is authenticated by the session module before any plaintext ever reaches this document's dispatch logic (Section **??**). This module never sees, and cannot be tricked by, an unauthenticated message – but that guarantee comes from a peer collaborator, not from a layer underneath it.

3 Command Set

3.1 Plaintext Command Envelope

The plaintext byte string delivered by the session layer (the decrypted content of a `CMD_SECURE_PAYLOAD` exchange) is structured as follows:

Field	Size	Description
OPCODE	1 byte	Identifies the command; see Table **??**
ARGS	variable	Opcode-specific arguments, may be zero-length

The corresponding response, returned to the session layer for encryption, uses the same shape with OPCODE replaced by a status byte (Section **??**).

3.2 Command Table

Command	OPCODE	Description
`CMD_UNLOCK`	`0x01`	Requests actuation of the lock mechanism to the unlocked position. ARGS: none (this revision).

Command	OPCODE	Description
CMD_LOCK	0x02	Requests actuation of the lock mechanism to the locked position. ARGS: none (this revision).
CMD_STATUS	0x03	Requests the current actuator state without changing it. ARGS: none.

Table 1: Application-layer opcodes defined in this revision.

An OPCODE not listed in Table **??** MUST be rejected with APP_STATUS_UNKNOWN_CMD (Section **??**) and MUST NOT reach the actuation logic in Section **??**. New commands (e.g. for OTA dispatch, Section **??**) are added by extending Table **??**, not by modifying the envelope format.

4 Authorization Model

This revision's authorization policy is intentionally simple, and is stated here explicitly rather than left implicit:

> **Authentication currently implies authorization.** Any device that successfully completes the session-layer handshake (i.e. proves possession of a private key corresponding to a provisioned Ed25519 public key) is permitted to issue any command in Table **??**.

There is no per-device role, scope, or permission check in this revision: this layer does not distinguish an owner's phone from a guest's phone, or an unlock request from a status query, for authorization purposes – both are governed by the same rule above. This is a conscious, reviewed scope limitation carried forward from the prior combined specification, not an oversight. Introducing differentiated authorization (an ACL, roles, or time-limited credentials) is deferred work; see Section **??**.

5 Lock Actuation

5.1 Actuator Abstraction Interface (AAI)

To keep this document independent of the specific motor/solenoid driver circuit, actuation is expressed against a small abstract interface, mirroring the pattern used for the transport layer's Link Layer Interface (LLI). Any conforming actuator driver MUST implement:

Primitive	Required Behavior
AAI_Engage()	Drives the mechanism to the unlocked position. Returns success/failure once motion completes or a hardware timeout elapses.
AAI_Disengage()	Drives the mechanism to the locked position. Same completion/timeout semantics as AAI_Engage().
AAI_GetState()	Returns the current mechanical state: LOCKED, UNLOCKED, or UNKNOWN (e.g. no position sensor fitted, or sensor fault).

The concrete driver behind the AAI (GPIO assignment, motor driver IC, timing) is out of scope for this document, matching the treatment of the LLI in *smartlock_transport_layer.pdf*.

5.2 Actuation Sequencing and Safety

1. `CMD_UNLOCK` / `CMD_LOCK` handling MUST call Section **??**'s authorization check before calling `AAI_Engage()` / `AAI_Disengage()`. There is currently no case in which the check can fail for an authenticated device (see Section **??**), but the call MUST remain in place so that future ACL enforcement (Section **??**) has a single insertion point.

2. Immediately before actuation, this layer SHOULD confirm (via whatever session-liveness signal the transport layer exposes) that the session is still active, to avoid actuating on behalf of a session that has already been torn down.

3. `AAI_GetState()` SHOULD be called both before and after actuation so the response to the Phone (Section **??**) reflects the mechanism's actual resulting state rather than an assumption that the drive command succeeded.

4. This revision does not define behavior for a mechanism that reports `UNKNOWN` state after actuation beyond returning `APP_STATUS_ACTUATOR_FAULT`; retry policy is left to a future revision.

6 Application-Layer Response Semantics

Transport-layer status words (SW1/SW2, defined in *smartlock_transport_layer.pdf* §8) only describe conditions visible to the transport and session layers (e.g. malformed APDU, handshake failure, auth-tag mismatch). They cannot describe application-layer outcomes, because by the time a status word is set the transport layer has no visibility into plaintext content. This layer therefore defines its own status byte, carried as the first byte of the encrypted plaintext response described in Section **??**:

Status	Value	Meaning
`APP_STATUS_OK`	`0x00`	Command executed (or, for `CMD_STATUS`, state read) successfully; actuator state follows in ARGS
`APP_STATUS_UNKNOWN_CMD`	`0x01`	OPCODE not present in Table **??**
`APP_STATUS_ACTUATOR_FAULT`	`0x02`	AAI reported `UNKNOWN` state after an actuation attempt

A transport- or session-layer failure (e.g. auth-tag mismatch, signature failure) never reaches this layer at all and therefore never produces one of these status bytes – those failures are signaled entirely below this layer, per *smartlock_transport_layer.pdf* §8 and *smartlock_session_layer.pdf* §10.

7 Collaboration with the Session Module

This is the same collaboration described from the other side in *smartlock_session_layer.pdf* §10; it is repeated here so this document is self-contained, not because the two directions are owned by different parties.

- **Session Module → Application Module:** this module receives only plaintext byte strings that the session module has already decrypted and authenticated. It MUST treat the sender's long-term identity as authenticated for every message it receives; it MUST NOT re-derive or re-check any cryptographic property – that would duplicate, not respect, the session module's responsibility.
- **Application Module → Session Module:** this module returns a plaintext byte string (status byte plus ARGS, Section **??**) for the session module to encrypt with the appropriate directional key. This module never has access to, and never handles, key material, nonces, or ciphertext directly – not because it sits below the session module, but because that is simply not this module's job.
- **No independent replay protection:** this module relies entirely on the session module's per-session key freshness (*smartlock_session_layer.pdf* §6.3) for replay resistance. It does not maintain its own sequence numbers in this revision.

8 Security Properties

Property	Mechanism	Notes
Authorization	Implicit – authentication equals authorization	Any authenticated identity is currently authorized for any command in Table **??**; see Section **??**
Actuation Integrity	Single-writer access to the AAI from the application task	No concurrent-actuation protection is defined beyond the transport layer's single-session-at-a-time model
Command Replay	Inherited from session layer key freshness	This layer adds no independent nonce or sequence number (this revision)
Command Injection	Strict OPCODE allow-list (Table **??**)	Unknown opcodes rejected before reaching Section **??**

9 Explicitly Deferred Items

The following items are acknowledged gaps, intentionally deferred rather than overlooked. Each is scoped here as a bounded extension point so that implementing it does not require re-splitting this document later – if any one of these grows large enough to warrant it, it SHOULD become its own sibling specification (following the same layering discipline used to separate this document from *smartlock_session_layer.pdf*) rather than being folded back into this file.

9.1 OTA Firmware Update Dispatch

Planned as one or more new opcodes in Table **??** (e.g. `CMD_OTA_BEGIN`, `CMD_OTA_CHUNK`, `CMD_OTA_COMMIT`). Open questions for a future revision:

- Whether OTA payloads route through the existing 227-byte unchained plaintext budget (*smartlock_transport_layer.pdf* §4.3) or require the transport layer to implement chaining, which is explicitly out of scope for that document's current revision.
- Firmware image authenticity (signing scheme, and whether it reuses the long-term Ed25519 identity key or a separate release key) – this is a session-layer or provisioning-layer concern and MUST NOT be designed inside this document.
- Whether OTA commands require a stricter authorization check than Section **??**'s current any-authenticated-device rule, which strongly motivates completing Section **??** first.

9.2 Access Control List (ACL) and Roles

Planned replacement for Section **??**'s current policy:

- An access control list of authorized Phone public keys per Lock, or a signed capability/credential issued by a backend authority.
- A revocation mechanism (e.g. denylist, short-lived signed credentials, or an online revocation check) so a lost or decommissioned phone can be removed without re-provisioning the Lock's entire trust store.
- Optional role/scope differentiation (e.g. owner vs. guest vs. time-limited access), which would extend Section **??**'s check to be per-command rather than all-or-nothing, and would give Section **??**'s insertion point (item 1) its first real failure case.

9.3 Out-of-Band BLE Command Source

Planned addition of BLE as a second transport carrying commands into this layer, alongside NFC. Open questions for a future revision:

- Whether BLE sessions reuse the existing session-layer handshake unchanged (preferred, since this layer's authorization model already only depends on "authenticated identity," not on which transport carried it), or require a BLE-specific pairing/bonding step layered underneath it.
- If reused, this document's Section **??** contract ("has no knowledge of transport") already accommodates a second transport with no changes required here – the work belongs in a new *smartlock_ble_transport_layer.pdf* sibling to *smartlock_transport_layer.pdf*, not in this document.
- Whether any command in Table **??** should be restricted to one transport only (e.g. disallowing `CMD_UNLOCK` over a longer-range BLE link for relay-risk reasons) is an authorization-policy question and belongs in Section **??**, not in a transport document.

10 Revision History

Version	Change	Date
0.1	**Initial standalone revision.** Split out of the combined *smartlock_application_layer.pdf* (v1.0), which previously mixed cryptographic session-establishment content (now *smartlock_session_layer.pdf* v1.1) with command dispatch, authorization, and lock-actuation content. Authorization Model content carried forward from that document's §9.2 essentially unchanged in substance. Newly defined in this revision: the plaintext command envelope and opcode table (Section **??**), the Actuator Abstraction Interface (Section **??**), and application-layer response status codes (Section **??**), none of which were specified in the prior combined document. Deferred-items section restructured around three named extension points (OTA, ACL, BLE) to bound future growth of this document.	July 24, 2026
0.2	**Reframed as a peer module, not a stacked layer.** Former "Relationship to Other Layers" and "Interface to the Session Layer" sections (which described this module as sitting "directly above" the session module and used "upward"/"downward" language) replaced by §**??** and §**??**, which present the session and application modules as equal-standing collaborators (Figure **??**). Made explicit that this module's actuator hardware dependency (Section **??**) is independent of, and parallel to, its session-module dependency – this module is not solely a communications consumer. Clarified that the session module does not own or control transport on this module's behalf. No functional changes to commands, authorization, or actuation logic.	July 24, 2026